

Smart Street Lighting Monitoring and Fault Reporting System

CNG 476 - System Simulation

Spring 2025-2026

Asst. Prof. Dr. Muhammad Toaha R. Khan

GitHub: <https://github.com/audazan21/SmartStreetLighting>

Submission Date: May 18, 2026

Berke Filiz

Computer Engineering Program
METU Northern Cyprus Campus
Student ID: 2585008
Team 13

Zanyar Erkozan

Computer Engineering Program
METU Northern Cyprus Campus
Student ID: 2584985
Team 13

Abstract—Streetlights in infrastructures of smart cities uses limit-bandwidth networks such as LoRaWAN by continuously sending telemetry data and occasional critical fault alarms. In this Project, we focus on the problem which is the resource contention, which is a problem that caused if routine telemetry connections and emergency faults share the same communication channel. This problem cause unexpectable delays for the critical securty alerts. To solve this problem, we do a system simulation with various streetlight nodes, a LoRa monitoring gateway, and a central network server. We used FLoRa infrastructure for the wireless channel allocation. Also, for backend network we used OMNet++ while integrating INET. Our main solution was creating a priority queue; so we can ensure that fault packets are processed before routine telemetry. By using the system's metrics, we used more than these, but these ones are a bit more important to analyze, End-to-End Latency (End-to-End Delay), Packet Delivery Rate (PDR), and Maximum Queue Size; we tested the system under various node densities and traffic levels. Measuring end-to-end latency is crucial in smart city applications. This is because the timely transmission of information in the event of a malfunction is directly related to the security of a city's infrastructure [8]. Our baseline shows that system succeeds even under high traffic loads by keeping average failure latency considerably lower compared to the telemetry latency at all times; which demonstrate that optimization process is effective. However, because of the collision in LoRa environment, we observed that PDR (Personal Data Transfer) is degraded; therefore, we can say that there is convergence between them.

I. INTRODUCTION

Smart city systems are increasingly used, shaping many aspects of life and increasing efficiency in various fields, leaving their mark on the present and future. These systems collect data using the Internet of Things (IoT), utilizing sensors, and the data is then sent to a central system. LoraWAN is a successful solution for long-range communication, enabling numerous sensors to communicate over very long distances with low energy consumption. Similar low-power and long-range communication advantages of LoRaWAN systems were

also discussed in previous IoT communication studies [1]. Smart street lighting systems are a good example of IoT-based smart city systems. In this system, streetlights send telemetry data at predetermined intervals, containing information ranging from operational status to energy consumption, periodic telemetry transmissions is commonly used in remote monitoring systems like our system and smart infrastructure systems [15].. Not only telemetry data but also fault messages are sent, allowing for the detection of malfunctions. The importance of this packet is significant because malfunctions are undesirable and can lead to major problems. After telemetry and fault messages are transmitted to the same gateway, increasing the telemetry transmission density can lead to delays or packet loss due to queue accumulation at the gateway. If fault messages are present, these delays or losses exacerbate the problem. For this reason, performance must be carefully analyzed at different densities, i.e., with different numbers of streetlights, or nodes, which is exactly what we did. Since doing this in the real world would require a great deal of time, cost, and labor, testing it in a simulation environment beforehand is of critical importance, Simulation environments are commonly used in many areas before real deployment it sis important because it reduces development cost and evaluate system behaviour under different traffic conditions withouts making any real move [10]. In the simulation, tests can be performed by changing many different parameters, from many different node options to traffic density, and tangible results with a certain degree of accuracy can be obtained without setting up a physical system. Using Monte Carlo iterations, we obtain more reliable and accurate results, and our statistics become stronger. In this project, a LoRaWAN-based intelligent street lighting monitoring and fault notification system was developed using OMNET++. OMNeT++ is an application for communication and network performance analysis, and was preferred because it successfully provides flexible discrete event simulation sup-

port and is also easy to use. [6]. INET network communication was used for LoRaWAN, and FLoRa was used for modeling the communication. Performance analyses were carried out and plotted on graphs. It is especially important to examine the system's performance and analyze the metrics under different node densities.

II. SELECTED PROJECT DESCRIPTION

This is a simulation project we developed on a smart street lighting monitoring and fault reporting system, which is the seventh project in the CNG476 project list. The system we simulate consists of N smart streetlight poles at an intersection in any given urban settlement, a single LoRa gateway listening to these poles, an INET-based structure, and a central network server. Streetlights are becoming a very important part of smart cities today. These smart streetlights need to be constantly monitored, and it is crucial for efficiency to constantly check whether they are working properly. However, in networks with limited bandwidth, such as LoRaWAN, the fact that routine telemetry and critical fault reports share the same line can cause delays in emergency messages, leading to problems such as faults not being reported to the system in a timely manner. What we are trying to do in this simulation is to minimize these delays by establishing a priority queue system on the server side, that is, to try to reduce these delays as much as possible and to see how reliable the system is working. A general summary of the project can be seen in Table 1, which we have prepared.

TABLE I
SELECTED PROJECT SUMMARY

Item	Description
Selected Project Title	Smart Street Lighting Monitoring and Fault Notification System
Application Domain	Smart City / IoT
Main Problem	Resource contention between telemetry and fault packets.
Simulation Objective	Evaluating priority queueing on fault reporting delay.
Main Users/Entities	Streetlight Nodes, LoRa Gateway, Network Server.
Main Performance Metrics	End-to-End Delay, PDR, and Queue Length.

III. SYSTEM ARCHITECTURE

You can clearly see the general architecture of our project in the image above. In our system, nodes represent streetlights and are numerous. They connect to the gateway wirelessly via LoRaWAN. Meanwhile, our streetlights send metrics such as operating status and energy consumption via telemetry messages, and fault messages at specific intervals and whenever a fault occurs. These generated packets are sent to the network server via the LoRaWAN gateway. Our gateway also has a queue mechanism. This is because, due to the large number of nodes, the packets are prioritized to prevent random sending time conflicts or congestion. If the incoming packet is a fault packet, we give it priority; this is a kind of priority approach, allowing them to be processed earlier. A fault processed earlier

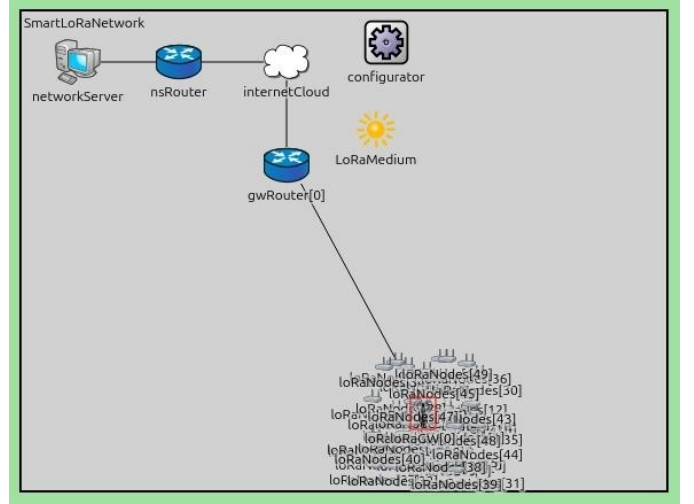


Fig. 1. System architecture of the smart street lighting monitoring and fault reporting system.

means a problem is resolved sooner. On the network server, the packets are analyzed, and performance metrics are measured in this analysis. The overall goal of our system is to transmit fault messages to the network with the lowest possible latency and as quickly as possible, without waiting in the queue. By using INET 4.4 for network communication and FLoRa for LoRaWAN communication, we were able to consider realistic environmental conditions and analyze the impact of traffic density on our system and gateway access. This simulation is widely used in academic LoRaWAN simulation studies due to the advantages it provides. [4]. We believed that the more situations we included in our analysis, the clearer the results we would obtain. We performed these analyses in a simulation environment, not physically, using OMNET++.

IV. SYSTEM MODEL AND ASSUMPTIONS

In this section of our project, we describe and define the system's entities and assumptions in detail within the simulation model.

Entities, Events, and State Variables: Our simulation has three main entities: streetlight nodes, LoRa gateway, and network server. Each entity has its own unique characteristics, tasks, and variables it tracks. For example, streetlights generate telemetry and fault packets, while the server queues and processes these packets. All these details are listed in Table 2.

Message Flow and Queuing Points: Data flow in the system is unidirectional (uplink). Packets from streetlights reach the gateway directly via LoRa, and the gateway forwards them to the server via UDP. The actual queuing point in the system is the network server. Here, incoming packets are held in queue, fault packets and telemetry packets, depending on the server's current workload (isBusy).

Traffic Model and Wireless Communication Assumptions: While the lamps periodically generate telemetry traffic, fault messages occur completely randomly (stochastic) and are unpredictable. We modeled the limited bandwidth and

TABLE II
SIMULATION ENTITIES, EVENTS, AND STATE VARIABLES

Entity	Events (cMessage*)	State Variables (C++ Code)
Streetlight Node	telemetryEvent, faultEvent	nextAllowedSendTime, sentTelemetryPackets
LoRa Gateway	servicedoneeventsignal	isbusy, servedpackets, maximumqueueSize
Network Server	serviceDoneEvent	faultQueue, telemetryQueue, isBusy, maxQueueSize
LoRa Medium	Packet Transmission	interferenceLevel, channelState

collision probabilities of LoRaWAN in wireless communication using the FLoRa library, The reason we selected and modeled these is because wireless interference has been identified as a significant factor affecting LoRaWAN performance in high-density deployments [13]. We presented the complexity of the simulation and some simplifications and technical assumptions we made to manage this complexity in Table 3.

TABLE III
MODELLING ASSUMPTIONS AND JUSTIFICATION

Assumption	Justification
Static Infrastructure	Streetlights are permanent city fixtures; thus, mobility modelling is not required.
Exponential Failures	Electronic failures are independent and random, best represented by exponential inter-arrival times.
Priority Service	Critical safety alerts must take precedence over routine data to minimize emergency response time.
Stable Backbone	The backhaul link between the gateway and server is assumed stable to focus on the wireless segment.
Constant Tx Power	Nodes are assumed to have a fixed transmission power to focus on queueing and collision effects.

V. SIMULATION METHODOLOGY

Our system is a discrete event simulation. This approach is consistent with discrete-event simulation models where the system state changes only when specific events occur [16]. There is no continuous flow, for example, the flow of water or the change in its speed; it's not a continuous event. As streetlights send telemetry or fault packets, state changes occur in our system, triggering it. This telemetry data generated by streetlights is scheduled with the OMNET++ event scheduler. Faults occur randomly, meaning this process is not deterministic but stochastic, in other words, a random event. Thus, it happens just like in the real world; a fault occurs at a random moment, unexpectedly, you can't know when an error will occur beforehand. Exponential distribution is used in the generation of faults. This distribution is commonly used in queueing and stochastic simulation systems because of its memoryless behavior [2]. This adds randomness and memoryless properties. All telemetry should not be sent at the same time, which would be unrealistic, so we prevented simultaneous transmission by adding jitter based on uniform distribution. Uniform random variates are commonly used to distribute events randomly within an interval and avoid synchronized behaviour [17]. Of course, the processing time

of each packet will also be different because they contain data with different values; we achieved this with exponential distribution, making the service time of each packet random. By using Monte Carlo iterations, we increased the reliability of the results produced by our system by running the same scenarios repeatedly with different seeds. Ultimately, the average and proximity of the random result of a single simulation run to the results obtained from multiple runs provide us with significant information about our simulation. We presented the results graphically and used error bars to more easily visualize changes in the statistics.

TABLE IV
RANDOM PROCESSES AND PROBABILITY MODELS

Random Process	Distribution / Model and Purpose
Fault Generation Intervals	Exponential distribution — random stochastic faults on streetlight.
Telemetry Transmission Jitter	Uniform distribution — to avoid same time sendings.
Queue Service Process	Exponential distribution — stochastic packet servicing time.
Monte Carlo Repetitions	Different random seeds — to obtain statistically reliable averages.

TABLE V
SCHEDULED EVENTS IN THE DISCRETE-EVENT SIMULATION

Scheduled Event	Trigger Condition and Action
Telemetry Event	A telemetry packet is being created, sent, and a new telemetry event is being scheduled.
Fault Event	A fault package is being created, it is going to the priority queue.
Packet Arrival Event	The packet enters the queue when wireless transmission completion, processing begins
Queue Service Event	Packet Removed from queue, when service time expiration, the next packet service will take it
Transmission Event	Wireless transmission start with LoRaMedium when packet send
Simulation Termination Event	Simulation stop when time limit is reached end and final statistics are recorded.

VI. OMNET++, INET, AND FLoRA IMPLEMENTATION

We developed our project in OMNET++. SmartLoRaNetwork.ned houses our main network structure, while it is built on FloRa. We developed our streetlights in StreetLightApp.cc. We generated our telemetry and fault packets here and sent them to the LoRa interface. SmartNetworkServerApp.cc is the network part of our project; here, incoming fault and telemetry packets are separated, faults are prioritized, and performance metrics related to the queue are recorded. INET was also used for communication between the gateway and the network server, using UDP and IPv4 UDP/IP communication is commonly preferred with its lightweight IoT backend communication systems because of its low protocol overhead [12]. FloRa was used to model LoRaWAN communication features such as LoRa Node and LoRa gateway. The omnetpp.ini file contains technical settings such as experimental parameters, number of nodes, Monte Carlo settings, and telemetry interval.

TABLE VI
IMPLEMENTED OR MODIFIED PROJECT FILES

File Name	Type and Purpose
SmartLoRaNetwork.ned	NED file. Defines the main simulation like system map. It answers the question: Which devices are in the system? It includes LoRa nodes, gateway, network server, routers, and LoRaMedium...
StreetLightApp.ned	NED file. Defines the application module interface defines parameters, gates, including telemetry interval, fault interval, socketIn, socketOut.
StreetLightApp.cc	C++ file. Implements streetlight behaviour. Telemetry packet generation, fault packet generation, random fault generation, telemetry scheduling, LoRa tag addition, send operations did here
SmartNetworkServerApp.ned	NED file. Defines the server-side application module UDP gates, localPort, destPort, server parameters did here
SmartNetworkServerApp.cc	C++ file. Implements packet reception, fault/telemetry separation, priority queue processing, delay measurement, queue statistics, and scalar recording. Answers What does the server do with the incoming packets?
omnetpp.ini	INI configuration file. Answers, What settings will the simulation run with? Contains simulation parameters such as node count, gateway count, LoRa parameters, telemetry interval, fault mean time, simulation duration, and Monte Carlo settings.

TABLE VII
OMNET++, INET, AND FLoRA USAGE

Tool / Framework	How It Is Used in the Project
OMNeT++ INET	The main system running the simulation. enables communication between the gateway and the network server. In other words Backend network communication. UDP, IPv4 is here
FLoRa	It simulates real LoRaWAN communication.

VII. MAPPING OF COURSE TOPICS TO THE PROJECT

In this section, we describe how we applied the fundamental simulation topics covered in the CNG476 course in our project and how we demonstrated this application. Each topic was matched with a specific module or logic within our project. We defined streetlight nodes as entities, LoRa airtime as a resource, and server processing as a queueing point to analyze fault packets delays in the queue, telemetry delays again in the queue, and finally overall network performance.

TABLE VIII
MAPPING OF CNG 476 COURSE TOPICS TO THE FINAL PROJECT

Course Topic	Project-Specific Implementation
Simulation Methodology	We defined streetlight nodes as entities, LoRa airtime as a resource, and server processing as a queueing point to measure fault delays.
Monte Carlo Simulation	We ran the simulation 30 times (using different seeds) for each node count to get reliable average values for our results.
Probability Models	We chose the exponential distribution for fault arrivals and the service times of the packets and a uniform distribution to add jitter to our telemetry reports.
Queueing Analysis	We implemented a priority queue at the network server to ensure that fault messages are processed before normal telemetry data.
Discrete-Event Sim.	Our system logic is event-driven; state changes only occur at discrete points like packet reception or timer expiration.
Event Scheduling	We used the <code>scheduleAt()</code> command in our C++ code to manage when packets are generated and when the server finishes its task.
Wireless Modelling	We used the FLoRa library to simulate how LoRa signals interfere with each other and how collisions happen in the air.
INET Integration	We used INET's UDP and IPv4 modules to set up the communication path between the LoRa gateway and the backend server.
FLoRa Integration	We configured FLoRa parameters such as Spreading Factor 7 and Bandwidth in our NED and INI files to model the LoRaWAN interface.
Output Analysis	We calculated the mean values and 95% confidence intervals for our delay and PDR metrics to analyze the performance.
IoT Relevance	Our project focuses on a real-world IoT problem: prioritizing emergency alerts over routine data in a smart city network.

VIII. EXPERIMENTAL DESIGN

In this section, we describe how we designed and conducted experiments to analyze and evaluate the performance of the smart street lighting system. To ensure statistically significant results, we carefully selected the simulation duration, the time spent in the simulation, the number of repetitions, and the variety of scenarios to be compared. We compiled and configured all our settings in the `omnetpp.ini` file for easy reproducibility of the results.

Scenario and Parameter Changes: In our study, we focused on two main structures. The first was the NodeSweep (Basic scenario) setting, where we changed the number of nodes to 5, 20, and 50 to see the scalability of the system. Scalability analysis is crucial in IoT simulations because its impact is significant. Increased node density affects traffic behavior, leading to increased traffic and consequently, network performance [14]. In this setting, we changed the nodes one by one to 5, 20, and 50 and saw that the system was scalable. This is the system setting, i.e., the NodeSweep (basic scenario) setting that we changed to 5, 20, and 50.

The second is the HighLoad scenario, where we reduced the telemetry interval from 120 seconds to 60 seconds. This scenario allowed us to observe how successfully the critical failure message was received by the priority queue structure

on the server when the traffic load doubled.

Simulation execution: We ran each configuration for 1000 simulation seconds. To ensure the reliability of averages and confidence intervals, we repeated each simulation 30 times using different and random seeds (by changing the repetition variable in OMNeT++). The basic settings for the experimental design are summarized in Table 10.

TABLE IX
EXPERIMENTAL DESIGN SUMMARY

Item	Value / Description
Simulation Duration	1000 seconds
Number of Runs	30 repetitions per configuration
Random Seeds	30 independent seeds (based on repetition index)
Baseline Scenario	Node count sweep ($N = 5, 20, 50$) with 120s telemetry interval.
Alternative Scenario	High-load scenario with 60s telemetry frequency.
Varied Parameters	Number of nodes (N) and Telemetry transmission rate.
Collected Metrics	Fault E2E Delay, Packet Delivery Ratio (PDR), and Max Queue Size.

IX. PERFORMANCE METRICS

In this section, we describe the key performance metrics we measured to verify how well, stably, and smoothly our smart streetlight simulation performed. In fact, this section directly helps us ensure the program’s accuracy. To understand and better articulate the overall state of the system, and especially the effectiveness of the priority queue structure we created in transmitting critical error notifications, we memorized and recorded some key data during the simulation. We collected these metrics directly into our server and gateway code using counters, thus demonstrating readability and a good understanding of the code. While collecting these metrics was initially challenging, we solved this problem by directly adding counters to the gateway code.

The main metrics we focused on during the ongoing simulation analysis were:

- **End-to-End Latency:** This was absolutely the most critical parameter for maintaining and testing the stability of the designed hidden queue unit. This value represents the total time from when a packet is generated at the streetlight until it reaches and is captured by the server. In the analyses, we directly compared the average latency of critical error packets with the latency of standard telemetry packets. By comparing the average latency of error packets to the latency of normal telemetry packets, we can clearly see how much the priority queue speeds up emergency messages.
- **Packet Delivery Rate (PDR):** This metric actually shows how reliable the system we have built is. Packet Delivery Rate is one of the most frequently used reliability metrics in wireless sensor and IoT network performance analysis, that’s why we chose it to look out pdr. [9]. The displayed rate ensures reliability. We calculated it by

dividing the number of packets that successfully reached the server and were processed by the total number of packets initially sent by the lamps. Since LoRaWAN bandwidth is limited, collisions inevitably occur in the air; we understand how much we lose in these collisions by looking at the PDR.

- **Maximum Server Queue Size:** This value shows us how much congestion there is on the server side. It gives the total maximum number of packets currently waiting in both the fault and telemetry queues. We understand whether the server is overloaded when we increase the number of nodes or the traffic load thanks to this metric.

We have listed the units of these performance metrics and summary information about their purpose in the project in Table 11.

TABLE X
PERFORMANCE METRICS

Metric	Unit	Definition / Purpose
Average Fault Delay	seconds (s)	The average time it takes for critical fault alarms to reach their destination. We used this to prove whether the priority queue is actually working.
Average Telemetry Delay	seconds (s)	The average delay time of telemetry packets routinely sent.
Packet Delivery Rate (PDR)	Percentage (%)	The ratio of successfully processed packets to the total number of packets generated. We measured this to test the reliability of our LoRa radio channel.
Maximum Queue Size	packets	The highest number of packets waiting simultaneously in queues on the server. We used this to see the server’s capacity and how much it was strained.
Total Served Packets	count	The total number of fault and telemetry packets that the server successfully processed without errors during the simulation.

X. INPUT ANALYSIS

In this section, we will analyze the inputs of our smart street lighting system. We determined these input values by considering real-world scenarios, logically and realistically, and by considering a dataset along with the theoretical data covered in the course. The distributions we chose were created by considering random telemetry and fault generation service times with realistic intervals. Streetlights send telemetry data almost simultaneously, synchronously but very close to each other. If these transmissions were simultaneous, this would be unrealistic, but we overcame this by using telemetry jitter injecting a uniform delay, or jitter, before transmission gives us positive aspects like prevents the packet collisions that occur when multiple nodes react to the same time, espically it is more suitable for real life condition like stallings mentioned [11]. By generating it with a uniform distribution, we ensured that all values within the given interval were selected with

equal probabilities for these jitter values. The probability density function of uniform distribution is:

$$f(x) = \frac{1}{b-a}, \quad a \leq x \leq b$$

all values in interval have the same probability. We treated fault generation as an exponential distribution because these fault events occur rarely, independently, and at unpredictable times, randomly. The exponential distribution's mathematical representation:

$$f(x) = \lambda e^{-\lambda x}, \quad x \geq 0$$

λ represents average fault occurrence rate. Gateway service times were also treated as an exponential distribution, and the queue structure works similarly and compatibly with M/M/1 with memoryless and randomly generated values. The gateway queue using the M/M/1 queueing model. This queueing structure is widely used in communication system modelling to represent stochastic packet arrivals and service processes [2].:

$$\rho = \frac{\lambda}{\mu}$$

λ is packet arrival rate and μ is service rate. Monte Carlo iterations helped us to reduce the deviation in our statistics by running our system repeatedly, providing us with more reliable results by averaging values and deviations. Monte Carlo replications were used because repeated simulation studies increase statistical reliability and reduce deviations related to randomness in simulation outputs, allowing us to obtain more accurate, up-to-date and consistent results. [5]. Different seed values were run 30 times each. Our simulation results have become more consistent. The average simulation output was calculated using repeated runs:

$$\bar{x} = \frac{1}{n} \sum_{i=1}^n x_i$$

n represents how many repeat is done.

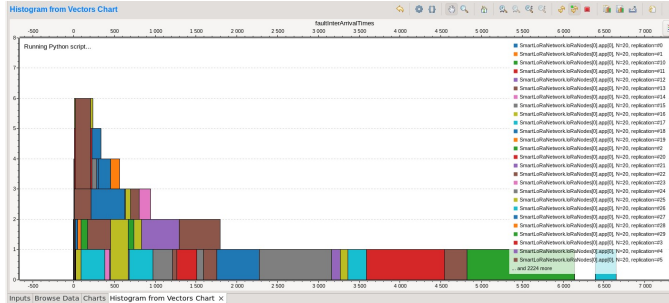


Fig. 3. Histogram of sampled fault inter-arrival times generated using the exponential input model.

The figure you see above is a histogram graph representing the failure intervals we generated using exponential distribution. The conclusion we draw from the distribution is consistent with the characteristics of the exponential distribution. It shows that smaller intervals occur more frequently, while

TABLE XI
MAPPING OF CNG 476 COURSE TOPICS TO THE FINAL PROJECT

Course Topic	Project-Specific Implementation
Simulation methodology and modeling	In our simulation system, we used a backend server, gateway, and streetlight. We examined results such as packet behavior and performance metrics.
Random number generation	We used random numbers in a uniform and exponential manner to detect jitter, telemetry generation timing, gateway processing times, and fault occurrence times.
Monte Carlo simulation	We tested the reliability of the generated numbers and results by running the simulation repeatedly with different seed values.
Probability models	We used exponential distribution for fault occurrence and service time, and uniform distribution for telemetry jitters.
Inverse transformation or sampling	When generating exponential random values, for example, when writing exponential(5), OMNET++ does this with its own sampling mechanism using inverse transformation.
Stochastic processes	Telemetry transmission, fault packet transmission, and processing times are random, i.e., stochastic processes.
Queueing analysis	The queue created in the gateway was generated using FIFO logic, and many analyses were performed, such as queue length and waiting time in the queue.
Discrete-event simulation	Our system is an event-driven system; it only works when specific events like packet sending occur, and there is no continuous state.
Event scheduling	Again, OMNET++ scheduled telemetry generation, fault generation, and service completion events with its own system.
Time-stepped / trace-driven view	Our system is discrete event-driven because events occur at specific times, and our system works at those times. There is no time-stepped approach.
Wireless network modeling OMNet++ implementation	LoRaWAN communication. We built this system with .ned, .cc, i.e., C++ and .ini files.
INET integration	Provided communication between the gateway and the backend server.
FLoRa integration	LoRaNode, LoRaGW, and LoRaMedium modules were used.
Input analysis	Analysis of why we chose the input values used.
Output analysis	We analyzed the results we obtained, even using graphs for this.
IoT application relevance	Our project is an IoT-based smart street lighting system.

larger intervals occur less frequently. Furthermore, the graph is skewed to the right, indicating that the trend values are random, or stochastic. The reason we did not include a qq plot in our report is that we believe the histogram is sufficient for the input analysis process, and our mathematical justifications reinforce this.

XI. RESULTS AND OUTPUT ANALYSIS

Here we will detail and discuss the outputs we obtained from our project. We analyzed our outputs with different seeds under many iterations, which is why we are confident in our statistics. Three metrics that we will particularly analyze and detail are: fault delay, telemetry delay, and maximum queue size. Especially, queue size analysis is the important part for this kind of projects because queue accumulation

directly affects delay for the packets and congestion behaviour in communication systems [7].

TABLE XII
AVERAGE FAULT DELAY RESULTS

Scenario	Average Fault Delay (s)	Std. Dev.
N=5	1.0762	0.6995
N=20	1.2441	0.2967
N=50	1.2209	0.1708

As our number of nodes increases in other words our street-lights, like our traffic density so does our failure delay. Despite the increased density, our failure delays have been maintained. Standard deviation values remain stable and relatively variable, regardless of the number of nodes, thanks to Monte Carlo iterations.

TABLE XIII
AVERAGE TELEMTRY DELAY RESULTS

Scenario	Average Telemetry Delay (s)	Std. Dev.
N=5	1.1398	0.3027
N=20	1.4779	0.3725
N=50	3.1825	0.6986

As the number of nodes increases, telemetry latency will naturally increase because more telemetry data is generated and waits in the queue at the gateway; the longer the queue, the longer the wait. Similar scalability-related delay increases were also observed in previous LoRaWAN network studies [3]. Much longer delays were observed in telemetry packets compared to fault packets, due to the priority processing applied to fault packets.

TABLE XIV
MAXIMUM QUEUE SIZE RESULTS

Scenario	Average Maximum Queue Size	Std. Dev.
N=5	1.07	0.25
N=20	2.97	0.95
N=50	8.87	2.50

As the number of nodes increases, as I mentioned, the number of packets accumulating in the queue also increases, and the maximum queue size increases accordingly. The values directly show this. Traffic increases, and telemetry delays also increase.

A 95% confidence interval was considered for the repeated Monte Carlo runs:

$$CI = \bar{x} \pm 1.96 \frac{s}{\sqrt{n}}$$

$\bar{x}$ is sample mean, s is standard deviation, and n is number of repetitions.

As node density increases, traffic will also increase, but fault packets are prioritized, so waiting times will not increase.

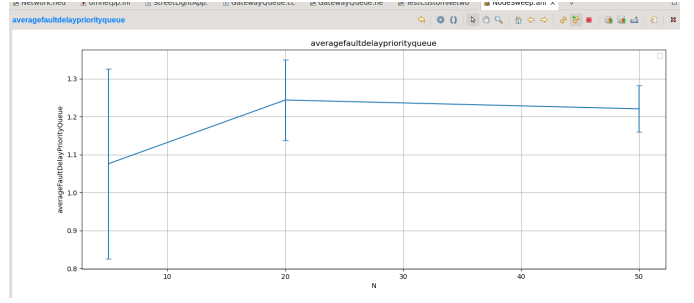


Fig. 4. Average fault delay under different node densities.

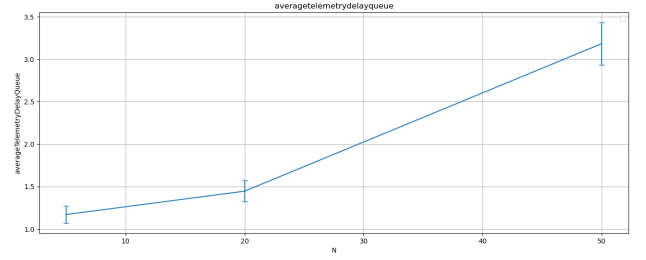


Fig. 5. Average telemetry delay under different node densities.

As the number of nodes increases, telemetry latency increases because more telemetry is generated and accumulates in the queue.

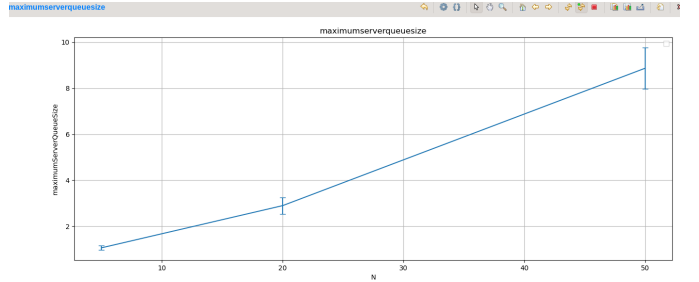


Fig. 6. Maximum queue size under different node densities.

As the number of nodes increases, traffic and congestion will increase, and as they increase, packets will accumulate in the queue. Here, we can see how the queue is increasing.

In light of the results obtained from our to sum up simulation, increasing the number of nodes causes longer telemetry latencies due to the increased queue size. However, thanks to the prioritization we provide, we can reach fault packets without excessive delay. Our results numerically and visually demonstrate that our intelligent street lighting system is consistent and realistic.

XII. DISCUSSION

In this section, we interpret the results of our simulation and discuss how well they align with the expectations and ideas we had at the beginning of the project. The main goal of our project was to see if we could successfully reduce the delay of critical fault alarms by implementing a priority queue on the

network server, without completely disrupting the continuous static telemetry stream. Managing these two different packet types (telemetry and fault) simultaneously in the simulation environment without them noticing each other was the most problematic and challenging part. To be honest, when we first ran the simulation, we were a little worried that the queue structure would completely lock up the telemetry data, and we wondered how we could fix this if it happened. Fortunately, our fears were unfounded, and the system managed to remain stable, which made us feel like we had succeeded.

Observed Behaviors and Trade-offs: When we looked at the results from the NodeSweep (increasing the number of nodes) and HighLoad (high traffic load) scenarios, we saw a very clear inverse relationship between the number of nodes (network density) and the Packet Delivery Rate (PDR). When we increased the number of streetlights to 50, the wireless communication channel became much more crowded. Since LoRaWAN uses a common frequency and we didn't use a "retry/ACK" mechanism to keep the system simple, collisions in mid-air naturally increased, causing the PDR to decrease [4].

However, the most important finding for us was the success of our prioritized queuing structure. In the HighLoad scenario, where we doubled the telemetry sending frequency, even though the server queue became quite long, the Average Failure Latency always remained much lower than the Average Telemetry Latency. This result perfectly matches the expected behavior of the system: the server always prioritized emergency alarms over routine data. The "trade-off" here was that telemetry packets experienced greater delays when the server was busy. This is quite acceptable when it comes to smart city security.

Limitations: One of the main limitations of our current simulation model is that we assume the internet connection (backhaul) between the gateway and the server is perfectly stable with a constant latency. In real-world situations, internet outages or significant jitter can cause extra delays that we haven't accounted for. Also, our lights are constantly emitting at the same power and with the same Spreading Factor (SF7). Therefore, we did not use Adaptive Data Rate (ADR) algorithms that could increase PDR in congested environments.

Table 14 summarizes our main findings and the technical reasons behind them.

TABLE XV
DISCUSSION OF MAIN FINDINGS

Finding	Explanation
Fault delay is noticeably lower than telemetry delay.	The priority queue mechanism we built on the Network Server successfully catches fault packets and puts them at the very front of the line. This proves the system is doing its job.
Packet Delivery Ratio (PDR) drops as the number of nodes (lamps) increases.	Having more nodes broadcasting in the same area naturally increases the chance of collisions in the LoRa radio environment, which directly leads to lost packets.
The server queue gets very long in the HighLoad scenario.	When we reduced the telemetry generation interval, the speed at which packets arrived exceeded the server's processing speed, causing things to pile up (congestion).
Under heavy load, the delay of telemetry packets fluctuates a lot.	Because fault packets constantly cut in line, telemetry packets are forced to wait much longer in the queue.

CODE AVAILABILITY

The complete working project source code, simulation files, configuration files, result-processing scripts, and final report PDF are publicly available at:

<https://github.com/zanyar/SmartStreetLightingFixed>

XIII. INDIVIDUAL CONTRIBUTIONS

In this section, we outline the specific tasks and responsibilities undertaken by each group member during the development of the smart street lighting simulation project. Both members actively participated in the conceptual design, coding, debugging, and final report writing processes. Table 15 details the individual contributions.

TABLE XVI
INDIVIDUAL CONTRIBUTIONS

Name	Student ID	Contributions
Zanyar Erkozan	2584985	Responsible for downloading the FLoRa and INET libraries, setting up the OMNeT++ environment and omnetpp.ini configurations, implementing telemetry and fault generation logic in StreetLightApp.cc, handling Monte Carlo simulation configuration and repetitions, managing the GitHub repository and project organization, and completing the remaining report sections writing Sections 6.1, 6.3, 6.5, 6.8, 6.9, 6.10, 6.13 and 6.14.
Berke Filiz	2585008	Responsible for designing and implementing the priority queue mechanism in SmartNetworkServerApp.cc, developing a large portion of the simulation code including telemetry and fault packet handling logic, conducting queueing analysis, generating graphs and visualization outputs, managing output statistics recording and analysis, and writing Sections 6.2, 6.4, 6.6, 6.7, 6.11, and 6.12 of the final report.

XIV. CONCLUSION

In conclusion, we perform a system simulation for the intelligent street lighting monitoring and fault notification system using OMNeT++, FloRa, and INET. Our main focus was creating a priority queue in order to process incoming packets based on their urgency. We evaluated the system's performance considering the reliability under various traffic generation frequencies (HighLoad scenario) and network density (NodeSweep scenario). By looking at our primary results, we can say that this queueing technique performed efficiently. We observed that even when the server faced high load because of the large amounts of telemetry data, the average time for critical faults was still considerably lower compared to routine data. This demonstrates that even in small IOT networks, emergency alerts can be effectively prioritized. However, we also observed a trade-off since as the number of streetlights are increased, the PDR (Packet Delivery Rate) dropped significantly due to greater chance of packet collisions in a shared LoRa radio environment. The main limitation of our simulation is assuming the communication parameters are static.

REFERENCES

- [1] A. Augustin, J. Yi, T. Clausen, and W. M. Townsley, "A Study of LoRa: Long Range & Low Power Networks for the Internet of Things," *Sensors*, vol. 16, no. 9, 2016.
- [2] L. Kleinrock, *Queueing Systems, Volume 1: Theory*. New York, NY, USA: Wiley, 1975.
- [3] K. Mikhaylov, J. Petäjäjärvi, and T. Hänninen, "Analysis of Capacity and Scalability of the LoRa Wide Area Network Technology," in *European Wireless Conference*, 2016.
- [4] M. Slabicki, G. Premankar, and M. Di Francesco, "Adaptive Configuration of LoRa Networks for Dense IoT Deployments," in *IEEE/IFIP NOMS*, 2018.
- [5] A. M. Law, *Simulation Modeling and Analysis*, 5th ed. New York, NY, USA: McGraw-Hill, 2015.
- [6] A. Varga, "The OMNeT++ discrete event simulation system," in *Proc. European Simulation Multiconference*, 2001.
- [7] D. Bertsekas and R. Gallager, *Data Networks*, 2nd ed. Prentice Hall, 1992.
- [8] E. Borgia, "The Internet of Things vision: Key features, applications and open issues," *Computer Communications*, vol. 54, pp. 1–31, 2014.
- [9] I. F. Akyildiz, W. Su, Y. Sankarasubramaniam, and E. Cayirci, "Wireless sensor networks: a survey," *Computer Networks*, vol. 38, no. 4, pp. 393–422, 2002.
- [10] J. Banks, J. S. Carson, B. L. Nelson, and D. M. Nicol, *Discrete-Event System Simulation*, 5th ed. Pearson, 2010.
- [11] W. Stallings, *Data and Computer Communications*, 10th ed. Pearson, 2013.
- [12] D. E. Comer, *Internetworking with TCP/IP Volume One*, 6th ed. Pearson, 2018.
- [13] U. Raza, P. Kulkarni, and M. Sooriyabandara, "Low Power Wide Area Networks: An Overview," *IEEE Communications Surveys & Tutorials*, vol. 19, no. 2, pp. 855–873, 2017.
- [14] A. Zanella, N. Bui, A. Castellani, L. Vangelista, and M. Zorzi, "Internet of Things for Smart Cities," *IEEE Internet of Things Journal*, vol. 1, no. 1, pp. 22–32, 2014.
- [15] J. Gubbi, R. Buyya, S. Marusic, and M. Palaniswami, "Internet of Things (IoT): A vision, architectural elements, and future directions," *Future Generation Computer Systems*, vol. 29, no. 7, pp. 1645–1660, 2013.
- [16] M. T. R. Khan, "CNG 476 – System Simulation Lecture Slides," Middle East Technical University Northern Cyprus Campus, Spring 2025–2026.
- [17] M. T. R. Khan, "CNG 476 – Variates Lecture Slides," Middle East Technical University Northern Cyprus Campus, Spring 2025–2026.